

Asignatura: Inteligencia Artificial

Período Académico: 2019-1

Grupo: IAI84-3 Jornada NOCHE

Programa: Ingeniería de sistemas.

Docente: Carlos Alejandro Trujillo Anaya

Fecha: 25 de Febrero de 2019.

ESTA ACTIVIDAD NO ES EVALUATIVA.

Actividad

Implementación de los algoritmos de búsqueda no informada **A*** y **Greedy search** (GS) en la solución de un problema búsqueda de ruta.

Objetivos

- Implementar un código en **Python** que lleva a cabo una búsqueda informada empleando la estrategia **A*** en la solución de un problema de búsqueda de ruta.
- Modificar el código suministrado para implementar la estrategia **GS**.
- A través de ejemplos básicos demostrar las diferencias, ventajas y desventajas de cada uno de los métodos.

Introducción

El problema de búsqueda de ruta consiste en encontrar el camino más corto entre dos puntos superando obstáculos. Lo anterior implica definir una posición inicial, una posición final y la posición de los obstáculos en una estructura de datos. Este problema es particularmente útil en la campo de la inteligencia artificial aplicada a los video juegos y a la robótica. Este problema sirve como base para la implementación de soluciones a problemas que involucren estrategias de búsqueda tanto informada como no informada.

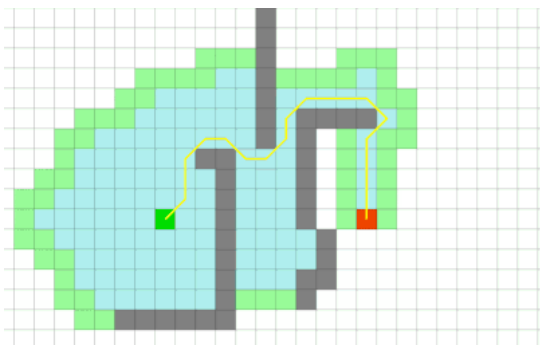


Fig. 1. Ilustración del problema de búsqueda de ruta.

Procedimiento

A 10x10 grid of dots. The letter 'T' is formed by dots in the top-left corner, and the letter 'S' is formed by dots in the bottom-right corner. The grid is sparse, with dots only appearing in the regions where the letters are located.

Alcaldía de Medellín
Cuenta con vos

Lee un archivo de texto y lo convierte en una lista.

def leerMapa(archivo):

 mapa = open(archivo, "r")

 mapa = mapa.readlines()

Quita el ultimo caracter de una lista.

 for i in range(len(mapa)):

 mapa[i] = mapa[i][: -1]

 for i in range(len(mapa)):

 mapa[i] = listarCadena(mapa[i])

 return mapa

Las funciones anteriores permiten leer el archivo de texto que contiene la información del mapa, organizar dicha información en una lista bidimensional, calcular la distancia entre dos puntos y llevar a cabo la búsqueda de un carácter dado en la lista.

Adicione la siguiente clase

class Mapa:

 def __init__(self, archivo):

 self.mapa = leerMapa(archivo)

 self.fil = len(self.mapa)

 self.col = len(self.mapa[0])

 def __str__(self):

 salida = ""

 for f in range(self.fil):

 for c in range(self.col):

 if self.mapa[f][c] == 0:

 salida += " "

 if self.mapa[f][c] == 1:

 salida += "# "

 if self.mapa[f][c] == 2:

 salida += "T "

 if self.mapa[f][c] == 3:

 salida += "S "

 if self.mapa[f][c] == 4:

 salida += ". "

 salida += "\n"

 return salida

 def camino(self, lista):

 del lista[-1]

 for i in range(len(lista)):

 self.mapa[lista[i][0]][lista[i][1]] = 4

Esta clase le permite crear objetos tipo mapa, tiene métodos auxiliares para la transferencia de la información del archivo de texto a una lista de Python, así como la demarcación del camino final encontrado por el método.

Agregue la siguiente clase:

```
class Nodo:
    def __init__(self, pos=[0, 0], padre=None):
        self.pos = pos
        self.padre = padre
        self.h = distancia(self.pos, pos_f)

        if self.padre == None:
            self.g = 0
        else:
            self.g = self.padre.g + 1
        self.f = self.g + self.h
```

Esta clase permite crear objetos relacionados con un estado (una posición en el mapa).

Finalmente, agregué la clase en la que se lleva a cabo el método de búsqueda **A***:

```
class AEstrella:
    def __init__(self, mapa):
        self.mapa = mapa

        # Nodos de inicio y fin.
        self.inicio = Nodo(buscarPos(2, mapa))
        self.fin = Nodo(buscarPos(3, mapa))

        # Crea las listas abierta y cerrada.
        self.abierta = []
        self.cerrada = []

        # Añade el nodo inicial a la lista cerrada.
        self.cerrada.append(self.inicio)

        # Añade vecinos a la lista abierta
        self.abierta += self.vecinos(self.inicio)

        # Buscar mientras objetivo no este en la lista cerrada.
        while self.objetivo():
            self.buscar()

        self.camino = self.camino()

    # Devuelve una lista con los nodos vecinos transitables.
    def vecinos(self, nodo):
        vecinos = []
        if self.mapa.mapa[nodo.pos[0]+1][nodo.pos[1]] != 1:
            vecinos.append(Nodo([nodo.pos[0]+1, nodo.pos[1]], nodo))
        if self.mapa.mapa[nodo.pos[0]-1][nodo.pos[1]] != 1:
```

```

        vecinos.append(Nodo([nodo.pos[0]-1, nodo.pos[1]], nodo))
    if self.mapa.mapa[nodo.pos[0]][nodo.pos[1]-1] != 1:
        vecinos.append(Nodo([nodo.pos[0], nodo.pos[1]-1], nodo))
    if self.mapa.mapa[nodo.pos[0]][nodo.pos[1]+1] != 1:
        vecinos.append(Nodo([nodo.pos[0], nodo.pos[1]+1], nodo))
    return vecinos

# Pasa el elemento de f menor de la lista abierta a la cerrada.
def f_menor(self):
    a = self.abierta[0]
    n = 0
    for i in range(1, len(self.abierta)):
        if self.abierta[i].f < a.f:
            a = self.abierta[i]
            n = i
    self.cerrada.append(self.abierta[n])
    del self.abierta[n]

# Comprueba si un nodo está en una lista.
def en_lista(self, nodo, lista):
    for i in range(len(lista)):
        if nodo.pos == lista[i].pos:
            return 1
    return 0

# Gestiona los vecinos del nodo seleccionado.
def ruta(self):
    for i in range(len(self.nodos)):
        if self.en_lista(self.nodos[i], self.cerrada):
            continue
        elif not self.en_lista(self.nodos[i], self.abierta):
            self.abierta.append(self.nodos[i])
        else:
            if self.select.g+1 < self.nodos[i].g:
                for j in range(len(self.abierta)):
                    if self.nodos[i].pos == self.abierta[j].pos:
                        del self.abierta[j]
                        self.abierta.append(self.nodos[i])
                        break

# Analiza el último elemento de la lista cerrada.
def buscar(self):
    self.f_menor()
    self.select = self.cerrada[-1]
    self.nodos = self.vecinos(self.select)
    self.ruta()

# Comprueba si el objetivo objetivo está en la lista abierta.
def objetivo(self):
    for i in range(len(self.abierta)):
        if self.fin.pos == self.abierta[i].pos:
            return 0

```

return 1

Retorna una lista con las posiciones del camino a seguir.

```
def camino(self):
    for i in range(len(self.abierta)):
        if self.fin.pos == self.abierta[i].pos:
            objetivo = self.abierta[i]

    camino = []
    while objetivo.padre != None:
        camino.append(objetivo.pos)
        objetivo = objetivo.padre
    camino.reverse()
    return camino
```

Una breve explicación de cada método se encuentra en el comentario que lo precede.

En este punto usted debería estar en capacidad de llevar a cabo la búsqueda bajo la estrategia **A*** para un mapa de búsqueda dado. Para lo anterior, puede implementar las siguientes instrucciones:

```
archivo="mapa.txt"
mapa = Mapa(archivo)
globals()["pos_f"] = buscarPos(3, mapa)
print mapa
A = AEstrella(mapa)
mapa.camino(A.camino)
print mapa
```

Al ejecutar el script anterior, se debería imprimir en consola el mapa inicial seguido del mapa con la ruta determinada por el método implementado.

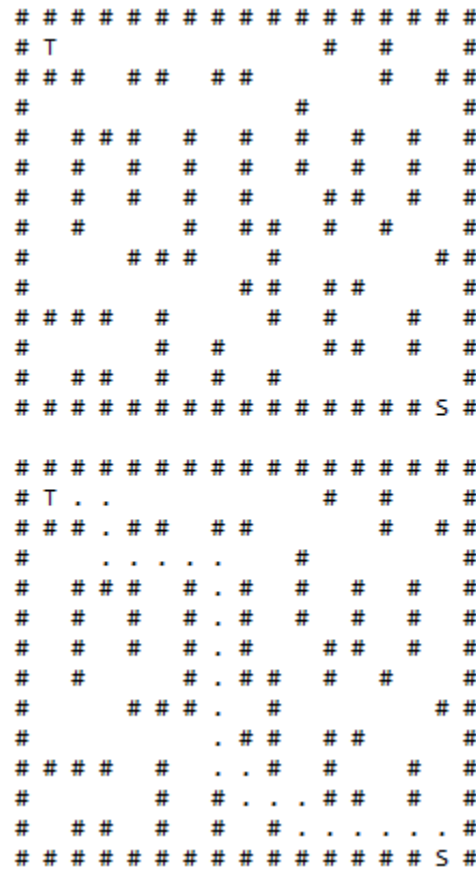


Fig. 3. Ejemplo de resultados de la búsqueda.

Una vez implementado y verificado el correcto funcionamiento de la estrategia **A***, modifique el código anterior para llevar a cabo una búsqueda del tipo **Greedy Search GS**. Recuerde que la diferencia entre ambos métodos consiste en la función de evaluación: **GS** solo considera la heurística, por el contrario **A*** considera tanto la heurística como el costo actual.

Para verificar la correcta implementación del método **GS**, emplee al menos dos mapas distintos en los cuales se puede observar la diferencia de caminos que arrojan las estrategias. Tenga en cuenta que **A*** es completa y óptima.

Contenido tomado de:

<http://razonartificial.com/2010/03/pathfinding-a-en-python-parte-i/>